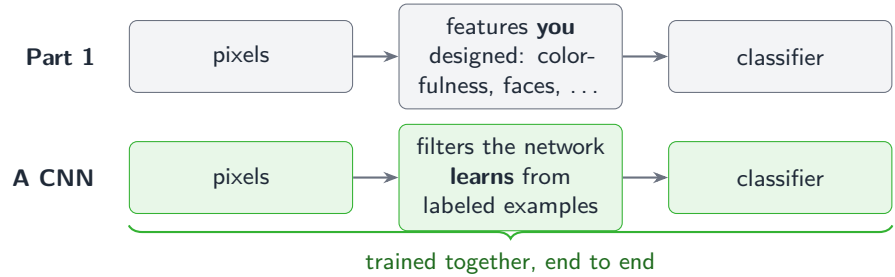
Pixels, Frames & Transformers

Convolutional Neural Networks for Images · Module 05

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

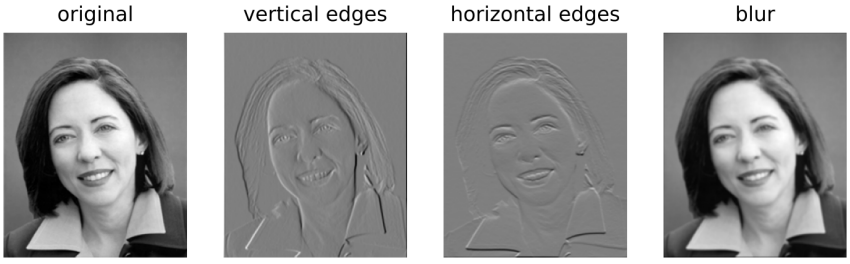
From Designed Features to Learned Features



The promise: features you would never think of designing, tuned to your task.

The price: the network has to learn every filter from examples, so it needs many labeled images.

Convolution: A Filter Sliding over Pixels



$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

the vertical-edge filter

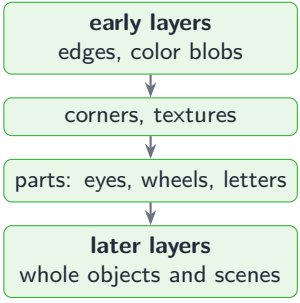
At every position the filter multiplies the pixels under it by its weights and adds them up. The result, a **feature map**, is bright wherever the image matches the pattern: here, dark next to bright, from left to right. Weights that sum to 0 ignore overall brightness and respond only to change.

Official portrait of Sen. Maria Cantwell, public domain, via unitedstates/images.

Pooling, and Layers That Build on Each Other

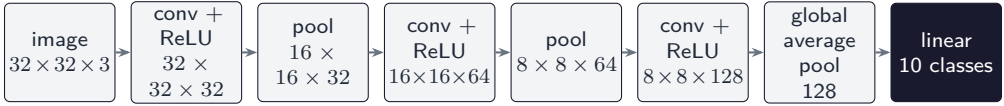


Max pooling keeps the largest value of every 2×2 block: the map gets four times smaller and a little less sensitive to exactly *where* something is.



Each layer combines the feature maps of the layer below. Nobody tells the network which patterns to look for.

A Small CNN, Layer by Layer



- About **95,000 weights**, almost all in the convolution filters
- Images get smaller (pooling) and "deeper" (more channels) as they pass through

Global average pooling reduces each feature map to one number, so the same network accepts a 32×32 image and a 96×96 one.

Training in One Loop

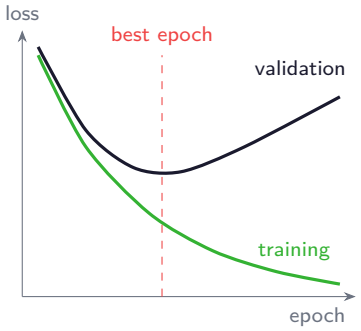
```
for x, y in train_loader:       # one batch
    logits = model(x)
    loss = loss_fn(logits, y)
    optimizer.zero_grad()
    loss.backward()             # gradients
    optimizer.step()    # nudge the weights
```

One pass over all training images is an **epoch**. PyTorch computes every gradient for you.

- **Training set**: what the weights learn from
- **Validation set**: watched after every epoch to see when to stop
- **Test set**: used once, at the end, for an honest estimate

On CIFAR-10 (50,000 labeled photos in ten classes), a network this small gets well beyond the 10% of guessing in a few minutes on a GPU.

Overfitting, and How to Fight It



schematic: the typical shape with too few images

With 1,000 images the network starts to **memorize** them: training loss keeps falling, validation loss turns around.

- **Augmentation**: a slightly different version of each image every epoch (crops, flips). Only changes that keep the label true.
- **Dropout**: randomly switch off part of a layer during training
- **Early stopping**: keep the weights from the best validation epoch

These help, but they stretch a small dataset; they don't replace a large one.

Looking Inside the Network

Filters
Every learned weight can be shown. First-layer filters look like color blobs and edges.

Activation maps
What each filter responds to in one particular image.

Saliency maps
The gradient of the class score with respect to each pixel: where a small change would matter most.

Saliency maps are hints, not explanations. Some popular methods give similar-looking maps even for a network with random weights (Adebayo et al., 2018). To show that a model relies on, say, the background, **test it**: replace the background and see whether the prediction changes.

Why Protest Photos Are Hard for a CNN from Scratch

Few labels
CIFAR-10 has 50,000; we have a few hundred, a few dozen of them King's Day.

A class defined by people and signs
in busy scenes, shrunk to 96 × 96 pixels, where signs become unreadable

Noisy labels
Commons categories say where an image was filed, not what it shows.

Variety
close-ups, crowds, aerial views; CIFAR-10 images are small and centered

Module 06: borrow a network trained on millions of images, and learn only the last step from your labels.

In the Notebook

- Filters and pooling by hand, on a portrait
- A small CNN in PyTorch, trained on CIFAR-10
- Overfitting on 1,000 images, then augmentation, dropout and early stopping
- Filters, activation maps and saliency maps, and their limits
- The same network on protest photos, scored on your 40 coded images
- Your scoreboard: comparing with module 04

part2_vectors/05_cnns_for_images/notebooks/05_cnns_for_images_exercises.ipynb · 6 exercises · GPU recommended